

RSA Public-Key Encryption and Signature Lab

Name: Ayah Husam Omar Sa'deh

ID: 0215258

Name: Kamelia Salameh Micheal Qumsieh

ID: 2212189

Course: Information and Network Security

Dr. Ramzi Saifan

Introduction

This lab introduces the principles and practical applications of RSA public-key encryption and digital signatures, two fundamental aspects of modern cryptography. RSA (Rivest-Shamir-Adleman) is a widely used cryptographic algorithm that ensures secure communication over the internet by utilizing a pair of mathematically related keys: a public key for encryption and a private key for decryption.

In this lab, we gain hands-on experience in implementing RSA key generation, encryption, decryption, signing, and signature verification using OpenSSL's BIGNUM library. Key concepts include:

- **Key Pair Derivation:** Calculating the private key from given prime factors p , q , and public exponent e .
- **Message Encryption and Decryption:** Applying modular arithmetic to secure and retrieve plaintext messages.
- **Digital Signatures:** Understanding how messages are signed with a private key and verified with a public key, ensuring message integrity and authenticity.
- **Certificate Verification:** Examining the cryptographic structure of X.509 certificates, a critical component of secure web communications.

Through practical tasks, this lab enhances understanding of modular exponentiation, hashing, and the vulnerabilities of improperly handled cryptographic operations. By the end of the lab, we appreciate the mathematical foundation and real-world applications of RSA, fostering **a deeper understanding of its role in securing digital communications.**

Task 1: Deriving the Private Key

In this task, the private key d for RSA encryption is calculated using the given prime numbers p , q , and public exponent e . The steps include:

1. Computing $n=p \times q$ (modulus for the public key).
2. Calculating $\phi(n)=(p-1) \times (q-1)$.
3. Finding d , the modular inverse of e modulo $\phi(n)$, satisfying $(e \cdot d) \bmod \phi(n)=1$.

The output includes p , q , n , $\phi(n)$, e , and d . This illustrates RSA key generation basics.

```
GNU nano 7.2
#include <stdio.h>
#include <openssl/bn.h>

// Helper function to print BIGNUM in hexadecimal
void printBN(const char *msg, const BIGNUM *a) {
    char *hex = BN_bn2hex(a);
    printf("%s %s\n", msg, hex);
    OPENSSL_free(hex);
}

int main() {
    // Initialize the BN_CTX structure
    BN_CTX *ctx = BN_CTX_new();

    // Create BIGNUM variables
    BIGNUM *p = BN_new();
    BIGNUM *q = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *d = BN_new();
    BIGNUM *phi = BN_new();
    BIGNUM *sub1 = BN_new();
    BIGNUM *sub2 = BN_new();

    // Assign values to p, q, and e
    BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
    BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
    BN_hex2bn(&e, "0D88C3");

    // Calculate n = p * q
    BN_mul(n, p, q, ctx);

    // Calculate phi = (p - 1) * (q - 1)
    BN_sub(sub1, p, BN_value_one()); // sub1 = p - 1
    BN_sub(sub2, q, BN_value_one()); // sub2 = q - 1
    BN_mul(phi, sub1, sub2, ctx); // phi = sub1 * sub2

    // Calculate d = e^(-1) mod phi
    BN_mod_inverse(d, e, phi, ctx);

    // Print results
    printBN("p =", p);
    printBN("q =", q);
    printBN("n =", n);
    printBN("phi(n) =", phi);
    printBN("e =", e);
    printBN("d (private key) =", d);

    // Free memory
    BN_free(p);
    BN_free(q);
    BN_free(n);
    BN_free(e);
    BN_free(d);
    BN_free(phi);
    BN_free(sub1);
    BN_free(sub2);
    BN_CTX_free(ctx);

    return 0;
}

aya@Ayah-Saadeh:~$ nano task1.c
aya@Ayah-Saadeh:~$ gcc -o task1 task1.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task1
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
n = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
phi(n) = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7981A4
e = 0D88C3
d (private key) = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
```

Task 2: Encrypting a Message

In this task, we encrypt the message "A top secret!" using the RSA public key. The message is first converted to its hexadecimal representation

(4120746f702073656372657421) and then transformed into a BIGNUM for encryption. Using the public key values $e=65537$ and n , the ciphertext is calculated as $C=M^e$. The task demonstrates how to apply RSA encryption in practice, ensuring secure transformation of plaintext into ciphertext. The provided private key can be used to verify the correctness of the encryption process.

```
#include <stdio.h>
#include <openssl/bn.h>

void printBN(char *msg, BIGNUM *a) {
    char *number_str = BN_bn2hex(a);
    printf("%s%s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main() {
    // Initialize the BIGNUM variables
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *M = BN_new();
    BIGNUM *C = BN_new();
    BN_CTX *ctx = BN_CTX_new();

    // Assign values to n, e, and M
    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&M, "4120746F702073656372657421");

    // Encrypt the message: C = M^e mod n

    BN_mod_exp(C, M, e, n, ctx);

    // Print the encrypted message

    printBN("Encrypted message: ", C);

    // Free the allocated memory
    BN_free(n);
    BN_free(e);
    BN_free(M);
    BN_free(C);
    BN_CTX_free(ctx);

    return 0;
}
```

```
aya@Ayah-Saadeh:~$ nano task2.c
aya@Ayah-Saadeh:~$ gcc -o task2 task2.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task2
Encrypted message: 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FADC
aya@Ayah-Saadeh:~$
```

Task 3: Decrypting a Message

In this task, the private key d is used to decrypt an encrypted message. The decryption process involves raising the ciphertext to the power of d modulo n to retrieve the original plaintext. This demonstrates the fundamental principle of RSA encryption and decryption, where the private key ensures secure message recovery.

```
#include <stdio.h>
#include <openssl/bn.h>

// Helper function to print BIGNUM in hexadecimal
void printBN(const char *msg, const BIGNUM *a) {
    char *hex = BN_bn2hex(a);
    printf("%s %s\n", msg, hex);
    OPENSSL_free(hex);
}

int main() {
    // Initialize the BN_CTX structure
    BN_CTX *ctx = BN_CTX_new();

    // Create BIGNUM variables
    BIGNUM *n = BN_new();
    BIGNUM *d = BN_new();
    BIGNUM *ciphertext = BN_new();
    BIGNUM *decrypted = BN_new();

    // Assign values to n, d, and ciphertext
    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");
    BN_hex2bn(&ciphertext, "8C0F971DF2F3672B28811407E2DABBE1DA0FE8BBDFC7DCB67396567EA1E2493F");

    // Decrypt the ciphertext: decrypted = ciphertext^d mod n
    BN_mod_exp(decrypted, ciphertext, d, n, ctx);

    // Print results
    printBN("Ciphertext (encrypted message) =", ciphertext);
    printBN("Decrypted message =", decrypted);

    // Free memory
    BN_free(n);
    BN_free(d);
    BN_free(ciphertext);
    BN_free(decrypted);
    BN_CTX_free(ctx);

    return 0;
}
```

```
aya@Ayah-Saadeh:~$ nano task3.c
aya@Ayah-Saadeh:~$ gcc -o task3 task3.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task3
Ciphertext (encrypted message) = 8C0F971DF2F3672B28811407E2DABBE1DA0FE8BBDFC7DCB67396567EA1E2493F
Decrypted message = 50617373776F72642069732064656573
```

The ASCII text of the decrypted text is:

```
aya@Ayah-Saadeh:~$ python3 -c 'print(bytes.fromhex("4120746f702073656372657421").decode("utf-8"))'
A top secret!
aya@Ayah-Saadeh:~$ python3 -c 'print(bytes.fromhex("50617373776F72642069732064656573").decode("utf-8"))'
Password is dees
```

Task 4: Signing a Message

This task involves digitally signing a message using a private key. We will sign the message "I owe you \$2000," modify it (e.g., change "\$2000" to "\$3000"), and compare the resulting signatures. This shows how even a small change in the message results in a completely different signature, highlighting the importance of message integrity in digital signatures.

```

aya@Ayah-Saadeh:~$ python3 -c 'print("I owe you $3000.".encode("utf-8"). hex())'
49206f776520796f752024333030302e
aya@Ayah-Saadeh:~$ python3 -c 'print("I owe you $2000.".encode("utf-8"). hex())'
49206f776520796f752024323030302e
aya@Ayah-Saadeh:~$

```

```

#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

// Helper function to print BIGNUM in hexadecimal
void printBNhex(const char *msg, const BIGNUM *a) {
    char *hex_str = BN_bn2hex(a);
    printf("%s %s\n", msg, hex_str);
    OPENSSL_free(hex_str);
}

int main() {
    // Initialize BN_CTX structure
    BN_CTX *ctx = BN_CTX_new();

    // Create BIGNUM variables
    BIGNUM *n = BN_new();
    BIGNUM *d = BN_new();
    BIGNUM *m2000 = BN_new();
    BIGNUM *m3000 = BN_new();
    BIGNUM *dc2000 = BN_new();
    BIGNUM *dc3000 = BN_new();

    // Assign values to n, d, m2000, and m3000 using hex values
    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");
    BN_hex2bn(&m2000, "49206f776520796f752024323030302e"); // "I owe you $2000."
    BN_hex2bn(&m3000, "49206f776520796f752024333030302e"); // "I owe you $3000."

    // Perform modular exponentiation to generate signatures
    BN_mod_exp(dc2000, m2000, d, n, ctx); // Signature for "I owe you $2000."
    BN_mod_exp(dc3000, m3000, d, n, ctx); // Signature for "I owe you $3000."

    // Print the digital signatures
    printBNhex("The digital signature of (I owe you $2000) =", dc2000);
    printBNhex("The digital signature of (I owe you $3000) =", dc3000);
}

```

```

aya@Ayah-Saadeh:~$ nano task4.c
aya@Ayah-Saadeh:~$ gcc -o task4 task4.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task4
The digital signature of (I owe you $2000) = 55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB
The digital signature of (I owe you $3000) = BCC20FB7568E5D48E434C387C06A6025E90D29D848AF9C3EBAC0135D99305822
aya@Ayah-Saadeh:~$

```

Task 5: Verifying a Signature

In this task, Bob verifies the authenticity of a message signed by Alice using her public key (e, n). He decrypts the signature using Alice's public key and compares it with the original message to check if they match. The task also explores the effect of a small modification in the signature, showing that even a single bit change causes the verification to fail.

```
aya@Ayah-Saadeh:~$ python3 -c 'print("Launch a missile.".encode("utf-8"). hex())'
4c61756e63682061206d697373696c652e
```

```
GNU nano 7.2 task5.c
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM as a hexadecimal string
void printBN(char *msg, BIGNUM *a) {
    char *number_str = BN_bn2hex(a);
    printf("%s%s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main() {
    // Initialize the BIGNUM variables
    BIGNUM *n = BN_new();
    BIGNUM *S = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *M = BN_new(); // The message we'll verify against
    BIGNUM *M_verify = BN_new(); // Decrypted signature
    BN_CTX *ctx = BN_CTX_new(); // Context for modular arithmetic

    // Assign values to e, n, and S
    BN_hex2bn(&e, "010001"); // Public exponent (65537)
    BN_hex2bn(&n, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115"); // Modulus n
    BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F"); // Signature S
    BN_hex2bn(&M, "4c61756e63682061206d697373696c652e"); // Message "Launch a missile."

    // Print the original message and signature
    printBN("Original message (M) = ", M);
    printBN("Signature (S) = ", S);

    // Verify the signature: M_verify = S^e mod n (decryption)
    BN_mod_exp(M_verify, S, e, n, ctx);

    // Print the decrypted signature
    printBN("Decrypted signature (M_verify) = ", M_verify);

    // Compare the computed M_verify with the original message M
    if (BN_cmp(M_verify, M) == 0) {
        printf("Signature verified: Message from Alice.\n");
    } else {
        printf("Signature verification failed: Message not from Alice.\n");
    }

    // Free the BIGNUM variables and context
    BN_free(n);
    BN_free(e);
    BN_free(M);
    BN_free(S);
    BN_free(M_verify);
    BN_CTX_free(ctx);

    return 0;
}
```

```
aya@Ayah-Saadeh:~$ nano task5.c
aya@Ayah-Saadeh:~$ gcc -o task5 task5.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task5
Original message (M) = 4C61756E63682061206D697373696C652E
Signature (S) = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
Decrypted signature (M_verify) = 4C61756E63682061206D697373696C652E
Signature verified: Message from Alice.
```

When altering the signature:

```
aya@Ayah-Saadeh:~$ nano task5.c
aya@Ayah-Saadeh:~$ gcc -o task5 task5.c -lssl -lcrypto
aya@Ayah-Saadeh:~$ ./task5
Original message (M) = 4C61756E63682061206D697373696C652E
Signature (S) = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F
Decrypted signature (M_verify) = 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
Signature verification failed: Message not from Alice.
```


Task 6: Manually Verifying an X.509 Certificate

Step 1: Download a certificate from a real web server: we choose to download the www.python.org website certificate, the result contains 2 certificates.

```
aya@Ayah-Saadeh:~$ openssl s_client -connect www.python.org:443 -showcerts
CONNECTED(00000003)
depth=2 OU = GlobalSign Root CA - R3, O = GlobalSign, CN = GlobalSign
verify return:1
depth=1 C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2
verify return:1
depth=0 CN = www.python.org
verify return:1
---
Certificate chain
 0 s:CN = www.python.org
  i:C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: May  9 12:50:11 2024 GMT; NotAfter: Jun 10 12:50:10 2025 GMT
```

```
-----BEGIN CERTIFICATE-----
MIIGeTCCBWGgAwIBAgIQAU36YGIvoQNeo/QVKzdBPjANBgkqhkiG9w0BAQsFADBY
MQswCQYDVQQGEwJCRTZMBcGA1UEChMQR2xvYmFsU2lnbiBudilzYTEuMCwGA1UE
AxMlR2xvYmFsU2lnbiBBdGxhcyBSMyBEVlBUTFMgQ0EgMjAyNCBMRmjaeFw0yNDA1
MDkxMjUwMTFaFw0yNTA2MTAxMjUwMTBaMBkxFzAVBgNVBAMMDnd3dy5weXRob24u
b3JnMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA3ojVsYzclvA49Nrm
Fh0WbRPGTP0+15K6+0mn3oYoVwYHfSJFncSaKOAcY/e3wptp+D26e7YXeA070Cum
KOKPhveTDWzGe+YAU2mEiLEGZvFAerQbfoLYV31phNzm4nziqxTHXT/KE2qdnK8Q
uULJCucRZGDUzQ02rXykhZ3GGFv9G9Yem8IJmHXZJGoIjjjdoXjHv5zd4f+K0t9m5
nClik+MxKbvQUgaSrJxRNetDfHhTWEoFPPFtFeQb5j9E0FkwZcX7N390TphUswQA
2S91i+EhCRxML/H2F/ZsGvyqIq5yIBIV9uG+04kKxutK73XRGJUJicrIw3xE/Xem
86eyCQIDAQABo4IDfDCCA3gwMwYDVR0RBcwwKoIod3d3LnB5dGhvb15vcmeCDCou
cHl0aG9uLm9yZ4IKcHl0aG9uLm9yZzA0BgNVHQ8BAf8EBAMCBaAwHQYDVR0lBBYw
FAYIKwYBBQUHAwEGCCsGAQUFBwMCMCB0GA1UdDgQWBwBBQySqsdlLNK11SyN55+cqgt
0vmXUTBxBgNVHSAEUDBOMAgGBmeBDAECATBCBgorBgEEAaAyCgEDMDQwMgYIKwYB
BQUHAgEwJmh0dHBzO18vd3d3Lmdsb2JhbHNpZ24uY29tL3JlcG9zaXRvcnkvaWAwG
A1UdEwEB/wQCAAAwGZ4GCCsGAQUFBwEBBBIGRMIGOMEAGCCsGAQUFBzABHjRodHRw
O18vb2NzcC5nbG91YXxzaWduLmNvbS9jYS9nc2F0bGFGZcjNkdnRsc2NhMjAyNHEy
MEoGCCsGAQUFBzACHj5odHRwO18vc2VjdXJlLmdsb2JhbHNpZ24uY29tL2NhY2Vy
dC9nc2F0bGFGZcjNkdnRsc2NhMjAyNHEyLmNydDAfBgNVHSMEGDAwBSvDQLQwxue
WPjvN2It5wNdnkFKijBIBgNVHR8EQTA/MD2gO6A5hdodHRwO18vY3J3Lmdsb2Jh
bHNpZ24uY29tL2NhL2dzYXR5YXNyM2R2dGxzY2EyMDI0cTIuY3J3SjMIIBfgYKKwYB
BAHWeQIEAgSCAW4EggFqAWgAdwDPEVbu1S58r/OHw9lpLpvpGnFnSrAX7KwB0lt3
zsw7CAAAAY9dZ3TYAAAEAwBIMEYCIQCaC0Mebu6Bj+14uCVpinvhkbn0nT2j1cbD
5VZx/B3FyGIhAIEBBxVAXEwQko4aEU2QYyeca90y0v6kCTtRHbBUwmojAHYAfVke
EuF4KnsCwYd8Xv340IdcFKB0LZ65Ay/ZDowuebgAAAGPXWd1twAABAMARzBFAiEA
ubj9nIgzRQG2uh4XiW6XfwyZ4Ey+gnqH+GrhmGnrY5cCIGSryEZqDXDhya06Ssm9
G74N7+1eKk6gSsy9vg+sVAY+AHUATnWjJ1yaEMM4W2zU3z9S6x3w4I4bjWnAsfPk
sWKA0d8AAAGPXWd2QgAABAMARjBEAiBjymqujRraDDgeDVspu7j89BNeh8E12n+
eQuXuZQghAigK8Q0DETPYKN34D7R0xFvBkSEs70TJHuKym9s1SQydAkWdQYJKoZI
hvcNAQELBQADggEBAAgyn+Kb0g0IB1tKZpSWGp5SSaNICPP34meQH+BQ3Gv74jjf
VZpILNaVJHImjDX3E5mvsOwAZN3/7YMO3hEjvtGf3kL35iYoQHRiYRJR89smOFM
a65kK6QUWVAHzu6EcSGCaid/sZzI5tNVH9yn00XPTTDBesh0ddy9dDutKyV+Bf02
U6UVuqezvm3nKdq3ytLB4C2KVMksHqerLM2N08TR1onbA6zzxqoDwuW2wHDB0pp1
ByrxC1lPWh44VJ0KsABLOHEd80e75/E7Mga0qxp0cPiUNLJk6s9RGQ5oUrnoyFE
sjBwG4C4CpCREc23DCyGPSDYshwQwuzUuDgiYsc=
-----END CERTIFICATE-----
 1 s:C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2
  i:OU = GlobalSign Root CA - R3, O = GlobalSign, CN = GlobalSign
```


9

Step 2: Extract the public key (e, n) from the issuer's certificate.

The obtained Modulus represents (n):

```
aya@Ayah-Saadeh:~$ openssl x509 -in c1.pem -noout -modulus
Modulus=8BE6F39245879FD1417872761FC627B6ECAC8A5B89383BE2B5F9193F8C
1C314A0F2CC254EA68336CCD734DC326D7320B1C9976D5CC15F3A3AFC11B01D1424FBD4F78D356513911912484E441717F373E6542EEB0917910DC6B33A489D9AA87FC9B0BB83DAAC28303DF214A46DB863EAC20D5EB
3654C3D8848DC8E8DBD16CA585CD9FAD7656132225188F1AFBB56DFDC6F60A6A34CBA2A1F357EDFBC44084D5F10C1BF5C667755C649604B101A7E2820FAB10253B1C10E0490688A03360E1B5B4A7ECFB910F2FA51C89B9C3
B9C3
aya@Ayah-Saadeh:~$
```

Modulus=8BE6F39245879FD1417872761FC627B6ECAC8A5B89383BE2B5F9193F8C
B5256027D1AA87EC2F18F4C7CEEB8987DB1FF7C5605DC2D945554BDCCBC31B5
0E5ECD070CCC3ECCD9FA8135A373E720EC91CD5E86B1C314A0F2CC254EA6833
6CCD734DC326D7320B1C9976D5CC15F3A3AFC11B01D1424FBD4F78D356513911
912484E441717F373E6542EEB0917910DC6B33A489D9AA87FC9B0BB83DAAC283
03DF214A46DB863EAC20D5EB3654C3D8848DC8E8DBD16CA585CD9FAD765613
2225188F1AFBB56DFDC6F60A6A34CBA2A1F357EDFBC44084D5F10C1BF5C6677
55C649604B101A7E2820FAB10253B1C10E0490688A03360E1B5B4A7ECFB910F2F
A51C89B9C3

```
aya@Ayah-Saadeh:~$ openssl x509 -in c1.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      80:4e:00:3a:27:2b:c5:18:e3:4d:a4:b1:fc:9b:78:33
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: OU = GlobalSign Root CA - R3, O = GlobalSign, CN = GlobalSign
    Validity
      Not Before: Jan 17 03:24:32 2024 GMT
      Not After : Jan 17 00:00:00 2026 GMT
    Subject: C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:8b:e6:f3:92:45:87:9f:d1:41:78:72:76:1f:c6:
        27:b6:ec:ac:8a:5b:89:38:3b:e2:b5:f9:19:3f:8c:
        b5:25:60:27:d1:aa:87:ec:2f:18:f4:c7:ce:eb:89:
        87:db:1f:f7:c5:60:5d:c2:d9:45:55:4b:dc:cb:c3:
        1b:50:e5:ec:d0:70:cc:c3:ec:cd:9f:a8:13:5a:37:
        3e:72:0e:c9:1c:d5:e8:6b:1c:31:4a:0f:2c:c2:54:
        ea:68:33:6c:cd:73:4d:c3:26:d7:32:0b:1c:99:76:
        d5:cc:15:f3:a3:af:c1:1b:01:d1:42:4f:bd:4f:78:
        d3:56:51:39:11:91:24:84:e4:41:71:7f:37:3e:65:
        42:ee:b0:91:79:10:dc:6b:33:a4:89:d9:aa:87:fc:
        9b:0b:b8:3d:aa:c2:83:03:df:21:4a:46:db:86:3e:
        ac:20:d5:eb:36:54:c3:d8:84:8d:c8:e8:db:d1:6c:
        a5:85:cd:9f:ad:76:56:13:22:25:18:8f:1a:fb:b5:
        6d:fd:c6:f6:0a:6a:34:cb:a2:a1:f3:57:ed:fb:c4:
        40:84:d5:f1:0c:1b:f5:c6:67:75:5c:64:96:04:b1:
        01:a7:e2:82:0f:ab:10:25:3b:1c:10:e0:49:06:88:
        a0:33:60:e1:b5:b4:a7:ec:fb:91:0f:2f:a5:1c:89:
        b9:c3
```

The Exponent represents (e):



```
Exponent: 65537 (0x10001)
X509v3 extensions:
  X509v3 Key Usage: critical
    Digital Signature, Certificate Sign, CRL Sign
  X509v3 Extended Key Usage:
    TLS Web Server Authentication, TLS Web Client Authentication
  X509v3 Basic Constraints: critical
    CA:TRUE, pathlen:0
  X509v3 Subject Key Identifier:
    AF:0D:02:D0:C3:1B:9E:58:F8:EF:37:62:2D:E7:03:5D:9E:41:4A:8A
  X509v3 Authority Key Identifier:
    8F:F0:4B:7F:A8:2E:45:24:AE:4D:50:FA:63:9A:8B:DE:E2:DD:1B:BC
  Authority Information Access:
    OCSP - URI:http://ocsp2.globalsign.com/rootr3
    CA Issuers - URI:http://secure.globalsign.com/cacert/root-r3.crt
  X509v3 CRL Distribution Points:
    Full Name:
      URI:http://crl.globalsign.com/root-r3.crl
  X509v3 Certificate Policies:
    Policy: 2.23.140.1.2.1
    Policy: 1.3.6.1.4.1.4146.10.1.3
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
  c6:b0:26:bf:04:8e:c5:39:68:b5:2f:44:cb:9c:4b:cc:c3:2b:
  df:8e:47:b2:ec:ea:5b:c9:bb:24:f4:71:9f:ce:c2:88:79:2d:
  04:21:7f:ae:2a:be:a7:b8:37:1b:79:fc:26:9b:7f:32:be:3c:
  d4:62:f9:c1:b3:4f:3a:45:2a:ac:ba:2c:cf:b3:e3:16:87:3e:
  9a:95:08:26:5d:99:f7:1c:9d:a2:41:f4:cc:15:97:b5:cd:bc:
  9a:4f:4c:ce:53:5c:18:a9:0d:65:4a:07:fb:68:20:0d:2b:92:
  33:c3:6d:10:7e:20:2d:e4:b2:c4:fe:fd:a9:30:d1:38:9f:41:
  d9:22:1e:13:dc:42:64:28:38:3a:46:52:45:bd:1b:9e:c7:af:
  a3:8b:0a:e9:b1:4c:46:1b:99:7b:5e:27:5d:f8:50:5d:41:7f:
  64:72:eb:e2:7a:a0:77:94:31:83:49:b8:67:d5:2e:9c:30:fe:
  a9:e5:13:8d:73:63:c0:9c:16:a9:f1:a6:e1:c7:21:ef:b7:38:
  db:da:fe:bf:ff:52:4a:38:7a:ee:9f:ee:37:cc:a7:d1:ca:64:
  dd:33:a7:6e:8c:08:3c:e9:5d:19:37:0e:5b:30:6f:e1:9d:cf:
  8c:26:80:42:ae:9d:49:00:99:7f:ea:1d:f5:df:5b:01:dc:59:
  d8:a0:79:94
```

Step 3: Extract the signature from the server's certificate

```
aya@Ayah-Saadeh:~$ openssl x509 -in c0.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      01:4d:fa:60:62:15:a1:03:5e:a3:f4:15:2b:37:41:3e
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = BE, O = GlobalSign nv-sa, CN = GlobalSign Atlas R3 DV TLS CA 2024 Q2
    Validity
      Not Before: May  9 12:50:11 2024 GMT
      Not After : Jun 10 12:50:10 2025 GMT
    Subject: CN = www.python.org
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:de:88:d5:b1:8c:dc:96:f0:38:f4:da:e6:16:1d:
        16:6d:13:c6:4c:fd:3e:97:92:ba:fb:49:a7:de:86:
        28:57:06:07:7d:22:45:9d:c4:9a:28:e0:1c:63:f7:
        b7:c2:9b:69:f8:3d:ba:7b:b6:17:78:0d:3b:38:2b:
        a6:28:e2:8f:86:f7:93:0d:6c:c6:7b:e6:00:bb:69:
        84:8a:51:06:66:f7:c0:7a:b4:1b:7e:82:d8:57:79:
        69:84:dc:e6:e2:7c:e2:ab:14:c7:5d:3f:ca:13:6a:
        9d:9c:af:10:b9:42:c9:0a:e7:11:64:60:d4:cd:0d:
        36:ad:7c:a4:85:9d:c6:18:5c:7d:19:87:a6:f0:82:
        66:1d:76:49:1a:82:23:8c:97:68:5e:31:ef:e7:37:
        78:7f:e2:8e:b7:d9:b9:9c:2d:62:93:e3:31:29:bb:
        d0:52:06:92:ac:9c:51:35:eb:43:7c:78:53:58:4a:
        05:3c:f1:6d:15:e4:1b:e6:3f:44:38:59:30:65:c5:
        fb:37:7f:74:4e:98:54:b3:04:00:d9:2f:75:8b:e1:
        21:09:1c:4c:2f:f1:f6:17:f6:6c:1a:fc:aa:22:ae:
        72:20:12:15:f6:e1:be:d3:89:0a:c6:eb:4a:ef:75:
        d1:18:95:09:89:ca:c8:c3:7c:44:fd:77:a6:f3:a7:
        b2:09
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Alternative Name:
        DNS:www.python.org, DNS:*.python.org, DNS:python.org
      X509v3 Key Usage: critical
        Digital Signature, Key Encipherment
      X509v3 Extended Key Usage:
        TLS Web Server Authentication, TLS Web Client Authentication
      X509v3 Subject Key Identifier:
        32:4A:AB:1D:2E:53:4A:97:54:B2:37:9E:7E:72:A8:2D:D2:F9:97:51
      X509v3 Certificate Policies:
        Policy: 2.23.140.1.2.1
        Policy: 1.3.6.1.4.1.4146.10.1.3
        CPS: https://www.globalsign.com/repository/
      X509v3 Basic Constraints: critical
        CA:FALSE
      Authority Information Access:
        OCSP - URI:http://ocsp.globalsign.com/ca/gsatlasr3dvtlsca2024q2
        CA Issuers - URI:http://secure.globalsign.com/cacert/gsatlasr3dvtlsca2024q2.crt
      X509v3 Authority Key Identifier:
        AF:0D:02:D0:C3:1B:9E:58:F8:EF:37:62:2D:E7:03:5D:9E:41:4A:8A
      X509v3 CRL Distribution Points:
```

```

Full Name:
URI:http://crl.globalsign.com/ca/gsatlasr3dvtlsca2024q2.crl
CT Precertificate SCTs:
Signed Certificate Timestamp:
  Version   : v1 (0x0)
  Log ID    : CF:11:56:EE:D5:2E:7C:AF:F3:87:5B:D9:69:2E:9B:E9:
              1A:71:67:4A:B0:17:EC:AC:01:D2:5B:77:CE:CC:3B:08
  Timestamp : May  9 12:50:12.312 2024 GMT
  Extensions: none
  Signature : ecdsa-with-SHA256
              30:46:02:21:00:9A:0B:43:1E:6E:EE:81:8F:E9:78:B8:
              25:69:8A:7B:E1:91:B9:F4:9D:3D:A3:95:C6:C3:E5:56:
              71:FC:1D:C5:CA:02:21:00:87:81:07:15:40:C4:4C:10:
              92:8E:1A:11:4D:90:61:87:9C:6B:DD:32:D2:FE:A4:09:
              3B:51:1D:B0:54:C2:6A:23
Signed Certificate Timestamp:
  Version   : v1 (0x0)
  Log ID    : 7D:59:1E:12:E1:78:2A:7B:1C:61:67:7C:5E:FD:F8:D0:
              87:5C:14:A0:4E:95:9E:B9:03:2F:D9:0E:8C:2E:79:B8
  Timestamp : May  9 12:50:12.535 2024 GMT
  Extensions: none
  Signature : ecdsa-with-SHA256
              30:45:02:21:00:B9:B8:FD:9C:88:33:45:01:B6:BA:1E:
              17:89:6E:97:7F:0C:99:E0:4C:BE:82:7A:87:F8:6A:E1:
              98:69:EB:63:97:02:20:64:AB:C8:46:6A:0D:70:E1:C9:
              A3:BA:4A:C9:BD:1B:BE:0D:EF:E9:5E:2A:4E:A0:4A:CC:
              BD:BE:0F:AC:54:06:3E
Signed Certificate Timestamp:
  Version   : v1 (0x0)
  Log ID    : 4E:75:A3:27:5C:9A:10:C3:38:5B:6C:D4:DF:3F:52:EB:
              1D:F0:E0:8E:1B:8D:69:C0:B1:FA:64:B1:62:9A:39:DF
  Timestamp : May  9 12:50:12.674 2024 GMT
  Extensions: none
  Signature : ecdsa-with-SHA256
              30:44:02:20:63:CA:6A:AE:8D:1A:DA:0C:38:1E:0D:5B:
              29:A6:EE:E3:F3:D0:4D:7A:1F:04:97:69:FE:79:0B:97:
              B9:94:20:84:02:20:2B:C4:34:0C:44:CF:60:A3:77:E0:
              3E:D1:3B:11:6F:06:44:84:B3:B3:93:24:7B:8A:62:6F:
              6C:95:24:32:74:09

```

Signature Algorithm: sha256WithRSAEncryption

Signature Value:

```

08:32:9f:e2:9b:3a:0d:08:06:5b:4a:66:94:96:1a:9e:52:49:
a3:48:08:f3:f7:e2:67:90:1f:e0:50:dc:6b:fb:e2:38:df:55:
9a:48:2e:76:95:24:72:26:8c:35:f7:13:99:af:72:c3:b0:01:
93:77:ff:b6:0c:3b:78:44:8e:fb:46:7f:79:0b:df:98:98:a1:
01:d1:21:84:49:f1:1f:6c:98:e1:4c:6b:ae:64:2b:a4:14:59:
50:07:ce:ee:84:71:21:82:6a:27:7f:b1:9c:c8:e6:d3:55:1f:
dc:a7:d3:45:cf:4d:30:c1:12:c8:4e:75:dc:bd:74:35:2d:2b:
25:7e:05:f3:b6:53:a5:15:ba:a7:b3:be:6d:e7:29:da:b7:ca:
d2:c1:e0:2d:8a:54:c9:2c:1e:a7:91:94:cd:8d:d3:c4:d1:d6:
89:db:03:ac:f3:c6:aa:03:c2:e5:b6:c0:70:c1:3a:9a:65:07:
2a:f1:0b:59:4f:5a:1e:38:54:9d:0a:b0:00:4b:38:71:1d:0f:
cd:1e:ef:9f:c4:ec:c8:1a:d2:ac:69:39:c3:e2:50:d2:c9:93:
ab:3d:44:64:39:a1:4a:e7:a3:21:44:b2:30:70:1b:80:b8:0a:
90:91:11:cd:b7:0c:2c:86:3d:20:d8:b2:1c:10:c2:ec:d4:b8:
38:22:62:c7

```


Signature Algorithm: sha256WithRSAEncryption

Signature Value:

```
Ayah-Saadeh:~$ cat signature | tr -d '[:space:]':  
08329fe29b3a0d08065b4a6694961a9e5249a34808f3f7e267901fe050dc6bfb238df559a482e76952472268c35f71399af72c3b0019377ffb60c3b78448efb467f790bdf9898a101d1218449f11f6c98e14c6bae642ba414595007ceee847121826a277fb19cc8e6d3551fdca7d345cf4d30c112c84e75dcdb74352d2b257e05f3b653a515baa7b3be6de729dab7cad2c1e02d8a54c92c1ea79194cd8dd3c4d1d689db03acf3c6aa03c2e5b6c070c13a9a65072af10b594f5a1e38549d0ab0004b38711d0fcd1eef9fc4ecc81ad2ac6939c3e250d2c993ab3d446439a14ae7a32144b230701b80b80a909111cdb70c2c863d20d8b21c10c2ecd4b8382262c7ayah
```

08329fe29b3a0d08065b4a6694961a9e5249a34808f3f7e267901fe050dc6bfb238df559a482e76952472268c35f71399af72c3b0019377ffb60c3b78448efb467f790bdf9898a101d1218449f11f6c98e14c6bae642ba414595007ceee847121826a277fb19cc8e6d3551fdca7d345cf4d30c112c84e75dcdb74352d2b257e05f3b653a515baa7b3be6de729dab7cad2c1e02d8a54c92c1ea79194cd8dd3c4d1d689db03acf3c6aa03c2e5b6c070c13a9a65072af10b594f5a1e38549d0ab0004b38711d0fcd1eef9fc4ecc81ad2ac6939c3e250d2c993ab3d446439a14ae7a32144b230701b80b80a909111cdb70c2c863d20d8b21c10c2ecd4b8382262c7

Step 4: Extract the body of the server's certificate

```
Ayah-Saadeh:~$ openssl asn1parse -i -in c0.pem  
0:d=0 hl=4 l=1657 cons: SEQUENCE  
4:d=1 hl=4 l=1377 cons: SEQUENCE  
8:d=2 hl=2 l= 3 cons: cont [ 0 ]  
10:d=3 hl=2 l= 1 prim: INTEGER :02  
13:d=2 hl=2 l= 16 prim: INTEGER :014DFA606215A1035EA3F4152B37413E  
31:d=2 hl=2 l= 13 cons: SEQUENCE  
33:d=3 hl=2 l= 9 prim: OBJECT :sha256WithRSAEncryption  
44:d=3 hl=2 l= 0 prim: NULL  
46:d=2 hl=2 l= 88 cons: SEQUENCE  
48:d=3 hl=2 l= 11 cons: SET  
50:d=4 hl=2 l= 9 cons: SEQUENCE  
52:d=5 hl=2 l= 3 prim: OBJECT :countryName  
57:d=5 hl=2 l= 2 prim: PRINTABLESTRING :BE  
61:d=3 hl=2 l= 25 cons: SET  
63:d=4 hl=2 l= 23 cons: SEQUENCE  
65:d=5 hl=2 l= 3 prim: OBJECT :organizationName  
70:d=5 hl=2 l= 16 prim: PRINTABLESTRING :GlobalSign nv-sa  
88:d=3 hl=2 l= 46 cons: SET  
90:d=4 hl=2 l= 44 cons: SEQUENCE  
92:d=5 hl=2 l= 3 prim: OBJECT :commonName  
97:d=5 hl=2 l= 37 prim: PRINTABLESTRING :GlobalSign Atlas R3 DV TLS CA 2024 Q2  
136:d=2 hl=2 l= 30 cons: SEQUENCE  
138:d=3 hl=2 l= 13 prim: UTCTIME :240509125011Z  
153:d=3 hl=2 l= 13 prim: UTCTIME :250610125010Z  
168:d=2 hl=2 l= 25 cons: SEQUENCE  
170:d=3 hl=2 l= 23 cons: SET  
172:d=4 hl=2 l= 21 cons: SEQUENCE  
174:d=5 hl=2 l= 3 prim: OBJECT :commonName  
179:d=5 hl=2 l= 14 prim: UTF8STRING :www.python.org  
195:d=2 hl=4 l= 290 cons: SEQUENCE  
199:d=3 hl=2 l= 13 cons: SEQUENCE  
201:d=4 hl=2 l= 9 prim: OBJECT :rsaEncryption  
212:d=4 hl=2 l= 0 prim: NULL  
214:d=3 hl=4 l= 271 prim: BIT STRING  
489:d=2 hl=4 l= 892 cons: cont [ 3 ]  
493:d=3 hl=4 l= 888 cons: SEQUENCE  
497:d=4 hl=2 l= 51 cons: SEQUENCE  
499:d=5 hl=2 l= 3 prim: OBJECT :X509v3 Subject Alternative Name  
504:d=5 hl=2 l= 44 prim: OCTET STRING [HEX DUMP]:302A820E777772E707974686F6E2E6F7267820C2A2E707974686F6E2E6F7267820A707974686F6E2E6F7267
```

```

504:d=5  hl=2 l= 44 prim:  OCTET STRING      [HEX DUMP]:302A820E777777E707974686F6E2E6F7267820C2A2E707974686F6E2E6F7267820A707974686F6E2E6F7267
550:d=5  hl=2 l= 14 cons:  SEQUENCE
552:d=5  hl=2 l=  3 prim:  OBJECT
557:d=5  hl=2 l=  1 prim:  BOOLEAN
560:d=5  hl=2 l=  4 prim:  OCTET STRING
566:d=5  hl=2 l= 29 cons:  SEQUENCE
568:d=5  hl=2 l=  3 prim:  OBJECT
573:d=5  hl=2 l= 22 prim:  OCTET STRING      [HEX DUMP]:301406082B0601050507030106082B06010505070302
597:d=5  hl=2 l= 29 cons:  SEQUENCE
599:d=5  hl=2 l=  3 prim:  OBJECT
604:d=5  hl=2 l= 22 prim:  OCTET STRING      [HEX DUMP]:0414324AAB1D2E534A9754B2379E7E72A82DD2F99751
628:d=5  hl=2 l=  87 cons:  SEQUENCE
630:d=5  hl=2 l=  3 prim:  OBJECT
635:d=5  hl=2 l= 80 prim:  OCTET STRING      [HEX DUMP]:304E3008060667810C0102013042060A2B06010401A0320A01033034303206082B06010505070201162668747470733A2F2F777772E6
76CF6F6261C7369676E2E636FD027265706F7369746F72792F
717:d=5  hl=2 l= 12 cons:  SEQUENCE
719:d=5  hl=2 l=  3 prim:  OBJECT
724:d=5  hl=2 l=  1 prim:  BOOLEAN
727:d=5  hl=2 l=  2 prim:  OCTET STRING      [HEX DUMP]:3000
731:d=5  hl=3 l= 158 cons: SEQUENCE
734:d=5  hl=2 l=  8 prim:  OBJECT
744:d=5  hl=3 l= 145 prim: OCTET STRING      [HEX DUMP]:30818E304006082B060105050730018634687474703A2F2F637302E676C6F6261C7369676E2E636FD02F63612F677361746C61737
2336476746C73636132303234713230406082B06010505073002863E687474703A2F2F7365637572652E676C6F6261C7369676E2E636FD02F63616365727472F677361746C617372336476746C73636132303234713
22E637274
892:d=4  hl=2 l= 31 cons:  SEQUENCE
894:d=5  hl=2 l=  3 prim:  OBJECT
899:d=5  hl=2 l= 24 prim:  OCTET STRING      [HEX DUMP]:30160014AF002D0C31B9E58FBEF37622DE7035D9E414A8A
925:d=4  hl=2 l= 72 cons:  SEQUENCE
927:d=5  hl=2 l=  3 prim:  OBJECT
932:d=5  hl=2 l= 65 prim:  OCTET STRING      [HEX DUMP]:303F303DA03BA0398637687474703A2F2F63726C2E676C6F6261C7369676E2E636FD02F63612F677361746C617372336476746C73636
13230323471322E63726C
999:d=4  hl=4 l= 382 cons: SEQUENCE
1003:d=5  hl=2 l= 10 prim:  OBJECT
1015:d=5  hl=4 l= 366 prim: OCTET STRING      [CT Precertificate SCTs
[HEX DUMP]:0482016A0168007700CF1156ED52E7CAFF3875D8692E9BE91A71674AB071ECAC01D25B77CECC3B080000018F5D6774D800000403004
830460221009A0B4311E06E818FE978B25698A7BE191BF949D3DA395CGC3E55671FC1DC5CA021008781071540C44C10928E1A114D9061879C6BD032D2FEA4093B511DB054C26A230076007D591E12E1782A7B1C616
775CFD0F800875314E04E959EB9032FD9068CE2799B80000018F5D6775B7000004030047304502100B9B8FD9C88334501B6BA1E17896E977FC09904CBE827A87F86AE19869EB639702064ABC8466A0D70E1C93BA4A
CA9BD1BBE0DEFE95E2A4E0404ACBDB0E0AC54063E0075004E75A3275C9A10C3385B6C40D3F3F52EB1DF0E08E188069C0B1FA6481629A39DF0000018F5D6776420000040300463044022063CA6AAE8D1AD0C381E0D5B2
9A6EEE3F30D0AD7A1F049769FE790B97B994208402202B4C34044CF60A377E03ED13B11F064484B38393247B8A626FC9524372409
1385:d=1  hl=2 l= 13 cons: SEQUENCE
1387:d=2  hl=2 l=  9 prim:  OBJECT
1398:d=2  hl=2 l=  0 prim:  NULL
1400:d=1  hl=4 l= 257 prim: BIT STRING

```

Hash of first certificate:

13f5c5878b0ae14e94e7e3c6dc4b62e7f5c8b6d198ebaf17e14ea0c04afadd90

```
aya@Ayah-Saadeh:~$ openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
aya@Ayah-Saadeh:~$ sha256sum c0_body.bin
13f5c5878b0ae14e94e7e3c6dc4b62e7f5c8b6d198ebaf17e14ea0c04afadd90  c0_body.bin
aya@Ayah-Saadeh:~$
```

Step 5: Verify the signature

We assign the values obtained from the previous steps to the variables in this code:

- (e) for the exponent
- (n) for the modulus
- (s) for the signature extracted from the server's certificate
- (m) for the body of the server's certificate.

Next, we decrypt the signature (s) and compare it with the decrypted body (m), which yields the same result.

[illegible]

The decrypted s

[illegible]


```

FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
003031300D06096086480165030402010500042013F5C5
878B0AE14E94E7E3C6DC4B62E7F5C8B6D198EBAF17E14EA0C04AFADD90

```

The decrypted truncated s

13F5C5878B0AE14E94E7E3C6DC4B62E7F5C8B6D198EBAF17E14EA0C04AFADD
90

The Certificate body

13F5C5878B0AE14E94E7E3C6DC4B62E7F5C8B6D198EBAF17E14EA0C04AFADD
90

The code:

```
GNU nano 7.2 task6.c
#include <stdio.h>
#include <openssl/bn.h>

#define BITS 256

void printBNhex(char *msg, BIGNUM *a) {
    // Use BN_bn2hex(a) for hex string
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main() {
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *decs = BN_new();

    // Hexadecimal values for n, e, s, and m
    BN_hex2bn(&n, "88E6F39245879FD1417872761FC627B6ECAC8A5B89383BE2B5F9193F8CB5256027D1AA87EC2F18F4C7CEE8B987DB1FF7C5605DC2D9455548DCCBC31B50E5ECD070CCC3ECCD9FA8135A373E72");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&s, "08329fe29b3a0d08065b4a6694961a9e5249a34808f3f7e267901fe050dc6bfbe238df559a482e76952472268c35f71399af72c3b0019377fffb60c3b78448efb467f790bdf9898a10d1d12184");
    BN_hex2bn(&m, "13f5c5878b0ae14e94e7e3c6dc4b62e7f5c8b6d198ebaf17e14ea0c04afadd90");

    // Perform modular exponentiation
    BN_mod_exp(decs, s, e, n, ctx);

    // Print the decrypted value
    printBNhex("The decrypted s", decs);

    // Mask the bits of the decrypted value to 256 bits
    BN_mask_bits(decs, BITS);

    // Print the truncated decrypted value
    printBNhex("The decrypted truncated s", decs);

    // Print the certificate body (n)
    printBNhex("The Certificate body", m);

    // Free the BIGNUMs and context
    BN_free(n);
    BN_free(e);
    BN_free(s);
    BN_free(m);
    BN_free(decs);
    BN_CTX_free(ctx);

    return 0;
}
```